

Authentication and Permission Policy

This document outlines how clients authenticate to the RAGenius Execution Subsystem and how execution permissions are defined and enforced. Proper authentication and authorization are critical to prevent unauthorized access to tools and to ensure that high-risk operations are not executed without consent.

Authentication

In production, every request to `/v1/*` endpoints must be authenticated. For the initial MVP, you may stub authentication behind environment configuration or API gateway, but a real implementation should support:

- **Service-to-service tokens:** Each application (e.g. `ragenius_app`, admin consoles) is issued an API token or JWT that must be presented in the `Authorization` header (`Bearer <token>`). Tokens should be short-lived and rotated regularly.
- **Scopes and roles:** Tokens carry scopes or claims indicating what operations the caller is allowed to perform (e.g. `execute_skill`, `list_skills`, `view_logs`). Scopes should map to permission policies to restrict tool use.
- **Tenant isolation:** For multi-tenant scenarios, tokens must include a tenant ID so that data and executions are isolated by tenant. Queries should always filter by tenant ID.
- **Secure storage:** API secrets and signing keys must be stored in environment variables or a secrets manager. They should not be checked into source control.

Recommended Implementation

1. **Issuer:** Use an identity provider (e.g. OAuth 2.0 server) that issues signed JWTs with custom claims.
2. **Verification:** The API layer must verify the signature, expiration, and audience of the token before processing the request.
3. **Authorization:** Extract scopes or roles from the token and enforce them at the route and tool permission level.
4. **Revocation:** Provide a mechanism to revoke tokens (e.g. by rotating signing keys or maintaining a token blacklist).
5. **Rate limiting:** Apply per-token rate limits to prevent abuse.

Permission Policy

The permission engine controls whether a tool call is allowed to run based on declared scopes and defined policies. There are four modes:

Mode	Meaning	Behavior
<code>auto_allow</code>	Low-risk operation	Execute without additional confirmation.

Mode	Meaning	Behavior
<code>restricted</code>	Allowed only under constraints	Execute only if conditions (e.g. environment, input) are met.
<code>require_confirmation</code>	Needs explicit confirmation	Pause execution and return a <code>pending_confirmation</code> result.
<code>blocked</code>	Not allowed	Do not execute; return a <code>permission</code> error.

Policy Definition

Permission policies can be stored in a database table with fields such as:

Field	Type	Description
<code>app_id</code>	string	Identifier of the calling application.
<code>tool_id</code>	string	Identifier of the tool.
<code>scope</code>	string	Permission scope (e.g. <code>external_api.write</code>).
<code>policy</code>	string	One of the modes above.
<code>conditions</code>	JSON	Optional conditions (e.g. allow only when <code>input.duration <= 60</code>).
<code>updated_at</code>	timestamp	When the policy was last updated.

Checking Permissions

Before invoking any tool, the permission engine performs the following steps:

1. **Lookup:** Find all policies where `app_id` matches the request's app ID and `tool_id` matches the tool being called.
2. **Select policy:** If multiple policies match, choose the most restrictive policy. For example, `require_confirmation` overrides `auto_allow`.
3. **Evaluate conditions:** If a `restricted` policy has conditions (e.g. maximum file size), evaluate them against the tool input. If conditions fail, treat as `blocked`.
4. **Enforce:**
5. If policy is `auto_allow`, proceed.
6. If `restricted` and conditions pass, proceed.
7. If `require_confirmation`, pause execution and return a `pending_confirmation` result with details of the tool and scope. The system must expose a confirmation API to resume execution once confirmation is provided.
8. If `blocked`, stop execution and return a `permission` error.

Confirmation Flow

When a `require_confirmation` policy is triggered, the execution is paused. The system returns a result like this:

```
{
  "execution_id": "exec_123",
  "status": "pending_confirmation",
  "result_type": "json",
  "result": {
    "required_confirmation": true,
    "tool_id": "mock_video_generation_tool",
    "permission_scope": "external_api.write"
  },
  "files": [],
  "errors": [],
  "logs_summary": "Execution paused because confirmation is required."
}
```

An external system (e.g. `ragenius_app`) should call a confirmation endpoint with the execution ID and a decision (approve or reject). Upon approval, the orchestrator resumes the workflow from the step that required confirmation. If rejected, the execution fails with a permission error. Confirmation tokens should have an expiration time (e.g. 15 minutes) after which the execution is automatically failed if no response is received.

Authorization of Admin Operations

Separate from execution, there are operations to register skills, register tools, list executions, inspect logs, and manage policies. These admin operations should be protected by roles (e.g. `admin`, `operator`) and their own scopes. For example:

- `manage_skills`: register and update skill manifests.
- `manage_tools`: register and update tools.
- `view_logs`: read execution logs and audit trails.
- `manage_permissions`: create or update permission policies.

Tokens that lack these scopes should not be able to perform these operations.

Best Practices

- Implement least privilege. Only grant the scopes required for the task.
- Deny by default. New tools or skills should not be callable until policies are explicitly defined.
- Version your permission policies. Track changes over time and audit who made modifications.
- Provide a secure UI or CLI for admins to manage policies.
- Log permission checks and decisions for auditing and debugging.

- Do not embed credentials or tokens in the permission policy definitions.
-